

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 1 – ANALYTICAL PROBLEM SOLVING

Slot-C

37

Course Code:	CSA1402	Course Title:	COMPILER DESIGN
CO Assessed:	CO1 – Imitation (L1)	Assessment:	Assessment Tool 1 – ANALYTICAL PROBLEM SOLVING
Weightage:	40% of CIA		
CO1 Statement:	Apply lexical analysis for token specification and recognition using LEX tool. (BL3)		
Student Name:	V. Vamsidhar Reddy	Reg. No.:	192365029
Section / Batch:		Date:	16/07/26

Code of Conduct

I, V. Vamsidhar Reddy / 192365029 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Vammi

Instructions

- Develop a modular and efficient Python program that satisfies all the functional requirements specified in the problem statement.
- Demonstrate the correctness of the implementation using suitable test cases and justify the output obtained.
- Follow good programming practices, including meaningful variable names, proper code indentation, comments, and error handling wherever applicable.
- Duration: 30 minutes.

Q. No	Problem Understanding (20)	Method Selection (20)	Solution Process (25)	Accuracy of Calculation (20)	Interpretation of Results (15)	Total Score (out of 100)
1	4 / 4	4 / 4	4 / 4	4 / 4	3 / 4	96.25
2	4 / 4	3 / 4	3 / 4	4 / 4	4 / 4	88.75
3	3 / 4	4 / 4	3 / 4	4 / 4	4 / 4	88.75
4	4 / 4	4 / 4	3 / 4	3 / 4	4 / 4	88.75
5	3 / 4	3 / 4	4 / 4	4 / 4	4 / 4	90
OVERALL RUBRICS VALUE						^
	/ 4	/ 4	/ 4	/ 4	/ 4	90.6
	/ 20	/ 20	/ 25	/ 20	/ 15	/ 100

N.D/A

ANALYTICAL PROBLEM SOLVING

1	Trace the processing of the input expression $P = (a * b) + (b * c) * 2$ through all phases of a compiler. a) Identify and list all tokens generated during lexical analysis. b) Construct the syntax tree during the parsing phase. c) Generate the intermediate code for the given expression. d) Apply suitable optimizations to the intermediate code. e) Explain the significance of the final target code prod
2	For each of the following Regular Expressions, construct the language. Write any FIVE valid strings for each Regular Expression. a) $(\epsilon + aa^*)^*$ b) $(a^* + b^*)^* abb$ c) $(a^* b^*)^* aba (a^* + b^*)^*$ d) $(a + ab)^*$ e) $(aba)^* + (a^* b^*)^*$
3	i) Find the number of tokens in the following C statement is <pre>main() { int a=10; char b ="abc"; in t c = 30; ch ar d = "xyz"; in /* comment */ t m = 40.5; }</pre> ii) Identify the lexical errors in the following C statement is <pre>void main() { int x=10, y=20; char * a; a= &x; x= 1xab; }</pre>
4	Find a regular expression corresponding to each of the following subsets of $\{0,1\}^*$: i) The language of all strings that have an even number of 0's. ii) The language of all strings that contain at least one 1. iii) The language of all strings in which both the number of 0's and the number of 1's are odd. iv) The language of all strings that start with 1 and end with 0. v) The language of all strings that do not contain consecutive 1's.

5 Consider a lexical analyzer for a simplified programming language. The language consists of the following tokens:

1. Keywords: if, else, while, return, int, float
2. Operators: +, -, *, /, =, ==, <, >
3. Separators: (,), {, }, :, ,
4. Identifiers: A sequence of letters followed by a sequence of digits, e.g., var123
5. Integer Literals: A sequence of digits, e.g., 12345
6. Floating-point Literals: A sequence of digits followed by a dot and another sequence of digits, e.g., 123.45

The lexical analyzer needs to tokenize the following input string:

`int var123 = 12345; float var456 = 123.45; if (var123 == var456) { return; }`

Questions:

- a) How many tokens are there in the given input string?
- b) Provide a list of tokens categorized by their types (keywords, operators, separators, identifiers, integer literals, floating-point literals).
- c) Assuming a finite state machine (FSM) is used for lexical analysis, estimate the number of state transitions required to tokenize the given input string. Provide your reasoning based on the transitions needed for each token type.

Compiler phases:-

Expression:

$$P = (a * b) + (b * c) * 2$$

(a) Tokens (Lexical Analysis):

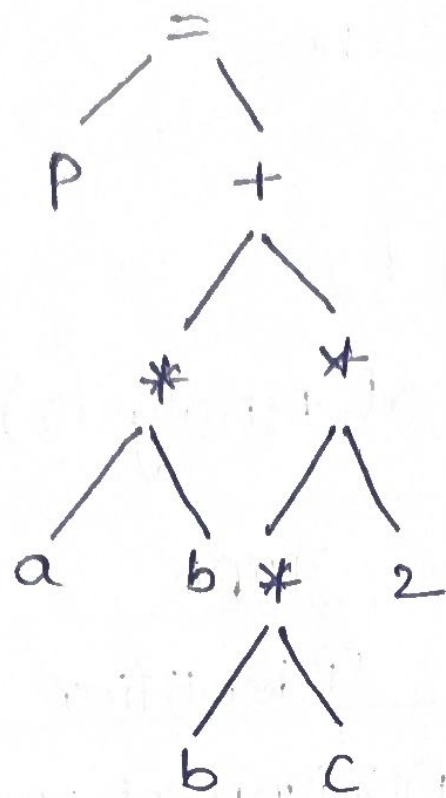
Token	TYPE
P	Identifier
=	Assignment operator
(	Left parenthesis
a	Identifier
*	Multiplication operator
b	Identifier
)	Right parenthesis
+	Addition operator
(	Left parenthesis
b	Identifier
*	Multiplication operator
c	Identifier
)	Right parenthesis
*	Multiplication operator
2	Integer constant

In Lexical Analysis,

Total Tokens = 15.

The expression contains 15 tokens.

(b) Syntax Tree :-



(c) Intermediate Code :-

$$t_1 = a * b$$

$$t_2 = b * c$$

$$t_3 = t_2 * 2$$

$$t_4 = t_1 + t_3$$

$$P = t_4$$

(d) Optimized Intermediate Code :-

$$t_1 = a * b$$

$$t_2 = b * c * 2$$

$$P = t_1 + t_2$$

(e) Significance of Target Code :-

Target code is the final machine/assembly code generated by the compiler. It can be executed directly by the processor and is optimized for efficient execution.

Regular Expression:-

①

$$(a) \underline{\epsilon aa^*}^* = (\epsilon + aa^*)^*$$

Language: Any number of groups of one or more a's.

Valid Strings:-

1. ϵ (empty string) - Epsilon

2. a

3. aa

4. aaa

5. aaaaaa

$$(b) \underline{(a^*b^*)^*}abb$$

Language: Strings ending with abb.

Valid Strings:-

1. abb

2. aabb

3. babb

4. aaabb

5. ababb

$$(c) \underline{(a^*b^*)^*}aba(a^*+b^*)^*$$

Language: Strings containing the substring aba.

Valid Strings:-

1. aba

2. abaa

3. abab

4. aaba

5. baaba

(d) $(aab)^*$

Language: Zero or more repetitions of aab.

Valid Strings:-

1. ϵ (empty string) - Epsilon

2. aab

3. aabaab

4. aabaabaab

5. aabaabaabaab

(e) $(aba)^* + (a^*b^*)^*$

Language: Union of $(aba)^*$ and $(a^*b^*)^*$

Valid Strings:-

1. ϵ (empty string) - Epsilon

2. aba

3. aaabbbb

4. aaaa

5. bbbb.

③ (i) Number of Tokens:-

Program:-

```
main()
```

```
{
```

```
int a = 10;
```

```
char b = "abc";
```

```
int c = 30;
```

```
char d = "xyz";
```

```
/* comment */ int m = 40.5;
```

```
}
```

Tokens:- ↓

②

main	=	"abc"	30	"xyz"	40.5
{	10	;	;	;	;
}	;	in	ch	in	}
{	char	t	an	t	-
int	b	c	d	m	-
a	=	=	=	=	-

∴ Total Token = 33

(ii): Lexical Errors:-

```
void main()
```

```
{
```

```
int x = 10; y = 20;
```

```
char *a;
```

```
a = 4x;
```

```
x = 1xab;
```

```
}
```

Errors:

1. 1xab →

Invalid hexadecimal constant

2. a = 4x; →

No lexical errors

(Only a semantic / type error).

④ Regular Expressions:-

(i) Even number of 0's:

→ $1^* (01^* 01^*)^*$

(ii) Atleast one 1:

→ $(0|1)^* (0|1)^*$

(iii) Odd number of 0's and odd number of 1's:

→ $1^* (01^* 01^*)^* 01^* (10^* 10^*)^* 0^*$

(iv) Starts with 1 and ends with 0:

→ $1(0|1)^* 0$

(v) No consecutive 1's:

→ $(0|10)^* 1?$

⑤ Lexical Analyzer:-

Input:-

Program-

```
int var123 = 12345;
```

```
float var456 = 123.45;
```

```
if (var123 == var456)
```

```
{
```

```
return;
```

```
}
```

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem
Method Selection	20%	Selects most appropriate method/formula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method
Solution Process	25%	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process
Accuracy of Calculation	20%	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations
Interpretation of Results	15%	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited unclear or interpretation	No interpretation